

Operatii cu fisiere

- Datele stocate in attributele unor obiecte din memorie, sunt **volatile**, se pierd la oprirea programului
- Pentru a stoca datele si dupa terminarea programului e nevoie ca ele sa fie tinute in fisiere, baze de date, etc.
- Java ofera clasa **File** pentru operatii cu fisiere dar nu pentru a citi sau scrie continutul fisierelor

Operatii cu fisiere

- **Constructori** pentru clasa `File`:
 - `File(String cale)`
 - `File(File director, String fisier)`
 - `File(String director, String fisier)`
- Obiectele de tip `File` pot descrie fisiere sau directoare

Operatii cu fisiere

- **Calea** catre un fisier reprezinta o modalitate de a specifica pozitia lui in sistemul de fisiere
 - **Absoluta**: porneste din radacina sistemului de fisiere pana la fisier
 - **Relativa**: porneste directorul curent pana la fisier
- In Unix se foloseste ca separator '/'
- In Windows se foloseste ca separator '\'

Operatii cu fisiere

- Exemple:

```
File f1 = new File("proiect",  
    "Student.java");
```

```
File f2 = new File("Profesor.java");
```

```
File f3 = new File("C:\\proiect\\n1");
```

```
File f4 = new File(f3, "Test.java");
```

Operatii cu fisiere

- Metode ale clasei File:

- ☐ **String getName()**
- ☐ **String getParent()**
- ☐ **String getAbsolutePath()**
- ☐ **boolean exists()**
- ☐ **boolean isFile()**
- ☐ **boolean isDirectory()**
- ☐ **boolean canRead()**
- ☐ **boolean canWrite()**

Operatii cu fisiere

- Metode ale clasei File:

- ☐ `long length()`
- ☐ `boolean delete()`
- ☐ `boolean mkdir()`
- ☐ `String[] list()` - afiseaza toate fisierele dintr-un subdir
- ☐ `File[] listFiles()` afiseaza toate fisierele si subdirectoarele dintr-un subdir

Operatii cu fisiere

- Este recomandat ca inainte de a citi sau scrie intr-un fisier sa se foloseasca un obiect de tip File pentru a **testa** daca acel fisier exista si daca avem drepturile de citire sau scriere
- Obiectele File sunt **finale**, nu pot descrie alt fisier decat cel specificat in constructor



Fisiere : Cuprins

- 1. Operatii cu fisiere. Clasa File**
- 2. Acces aleator la fisiere**
- 3. Fluxuri de date**
- 4. Serializarea obiectelor**

Acces aleator

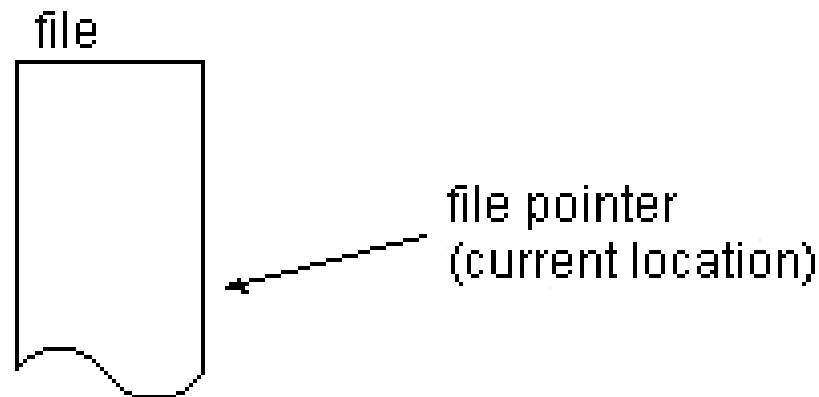
- Accesul aleator la un fisier inseamna ca putem citi sau scrie in fisier la orice pozitie
- Acest lucru se poate realiza in Java prin obiecte din clasa **RandomAccessFile**

```
new RandomAccessFile("date", "r");
```

```
new RandomAccessFile("test", "rw");
```

Acces aleator

- Dacă fișierul nu există și modul de deschidere a fost “r” (doar citire), atunci se aruncă o excepție **FileNotFoundException**
- Dacă fișierul nu există dar modul de deschidere a fost “rw” (citire și scriere), atunci se creează un fișier de lungime zero
- Clasa `RandomAccessFile` suportă noțiunea de *pointer de fișier*. Acesta este un indicator ce specifică poziția curentă în fișier.
- La deschiderea unui fișier pointerul are valoarea 0, indicând începutul fișierului.



Acces aleator

- Clasa **RandomAccessFile** ne ofera metode pentru a citi si a scrie in fisier date de orice tip primitiv si siruri de caractere Unicode (UTF – Unicode Transformation Format)
- Ex:
 - **boolean readBoolean()**
 - **void writeBoolean(boolean b)**
 - **char readChar()**
 - **void writeChar(char c)**
 - **int readInt()**
 - **void writeInt(int i)**
 - **String readUTF()**
 - **void writeUTF(String s)**
 - Apeluri la metode readXXX sau writeXXX deplaseaza pointerul fisierului cu numarul de octeti cititi sau scrisi de metodele respective.

Acces aleator

- In plus fata de metodele de citire/scriere clasa pune la dispozitie si metode pentru controlul pozitiei pointerului de fisier. Acestea sunt:
- `int skipBytes ( int n )` Muta pointerul fisierului înainte cu un numar specificat de octeti
- `void seek (long pozitie)` Pozitioneaza pointerului fisierului înaintea octetului specificat.
- `long getFilePointer ( )` Returneaza pozitia pointerului de fisier (pozitia de la care se citeste/la care se scrie)

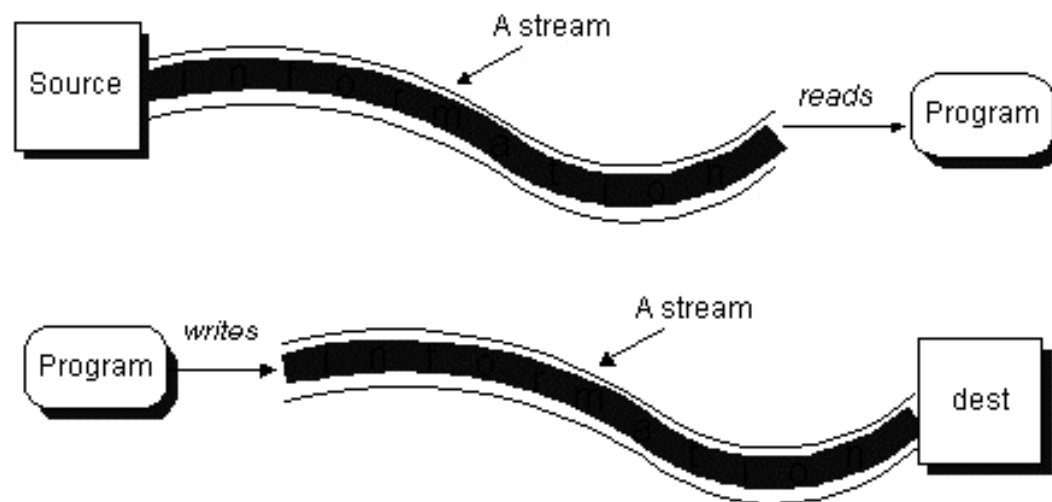


Fisiere : Cuprins

- 1. Operatii cu fisiere. Clasa File**
- 2. Acces aleator la fisiere**
- 3. Fluxuri de date**
- 4. Serializarea obiectelor**

Fluxuri de date

- Toate clasele pentru lucrul cu fisiere se gasesc in pachetul **java.io**
- Exista doua abordari in lucrul cu fisiere:
 - Accesul aleator la fisiere
 - Tratarea fisierelor ca un flux de date
- Intr-un flux datele pot curge intr-o singura directie
- Orice flux de date are o sursa (care poate doar scrie date in flux) si o destinatie (care poate doar citi date din flux)



Fluxuri de date

- Un *flux* este un canal de comunicație unidirecțional între doua procese.
- Un proces care descrie o sursa externa de date se numeste *proces producator*.
- Un proces care descrie o destinatie externa pentru date se numeste *proces consumator*.
- Fluxurile de date pot fi impartite in doua categorii, dupa pozitia programului fata de flux:
 - **Fluxuri de intrare** – programul este destinatia fluxului iar sursa este un fisier, un array, un alt program
 - **Fluxuri de iesire** – programul este sursa fluxului iar destinatia este un fisier, un array sau un alt program

Fluxuri de date

- Observatii:
- Fluxurile sunt canale de comunicatie seriale pe 8 sau 16 biti.
- Fluxurile sunt unidirectionale, de la producator la consumator
- Fiecare flux are un singur proces producator si un singur proces consumator
- Intre doua procese pot exista oricâte fluxuri, orice proces putând fi atât producator si consumator în acelasi timp, dar pe fluxuri diferite
- Consumatorul si producatorul nu comunica direct printr-o interfata de flux ci prin intermediul codului Java de tratare a fluxurilor

Clasificarea fluxurilor

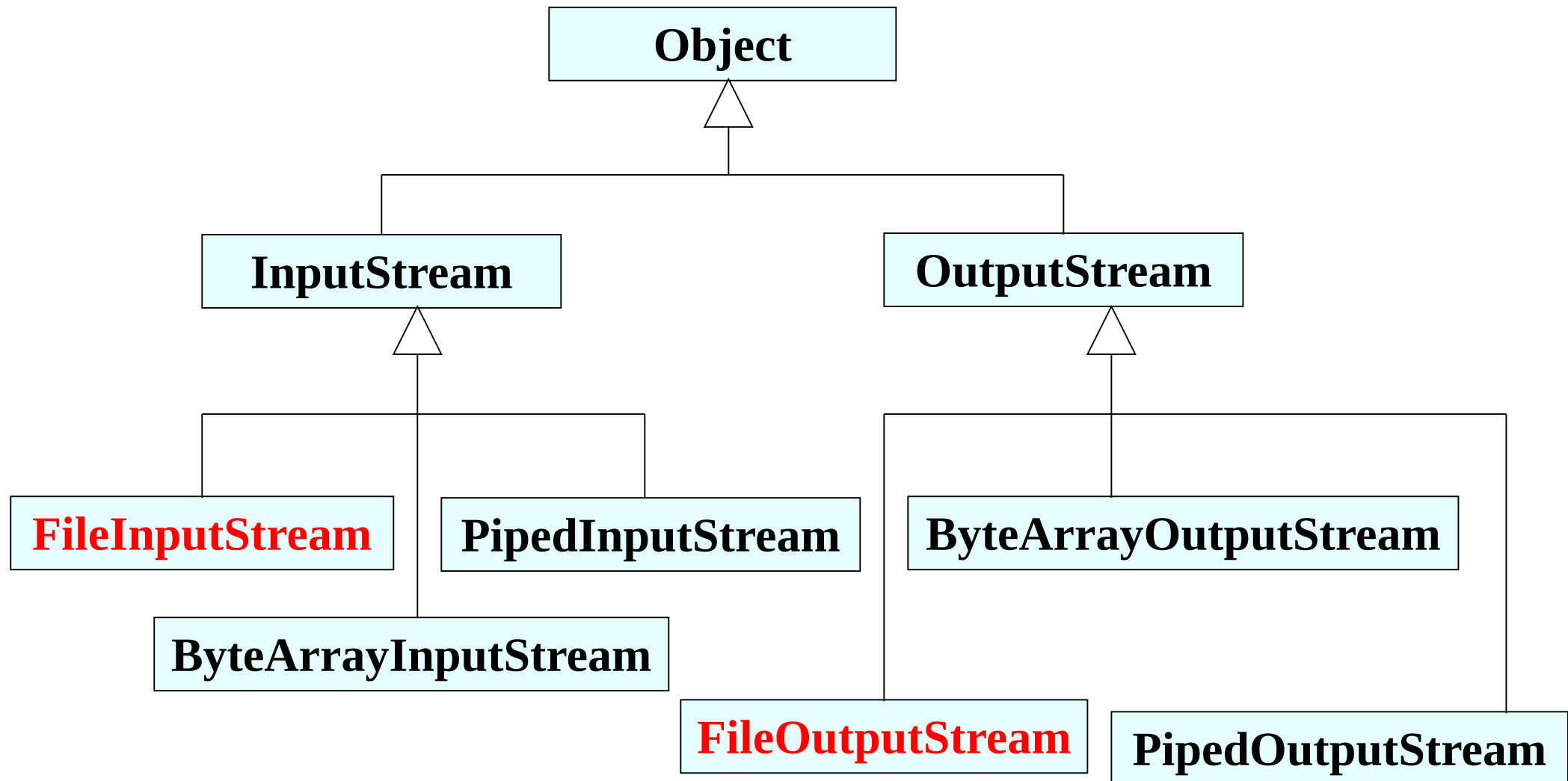
- Exista trei tipuri de clasificare a fluxurilor:
- Dupa "directia" canalului de comunicatie deschis fluxurile se împart în
 - fluxuri de intrare (pentru citirea datelor)
 - fluxuri de iesire (pentru scrierea datelor)
- Dupa tipul de date pe care opereaza:
 - Fluxuri de octeti – lucreaza cu informatie la nivel de octeti (comunicația serială pe 8 biți), sunt derivate din clasele InputStream si OutputStream
 - Fluxuri de caractere – lucreaza cu caractere format Unicode (comunicația serială pe 16 biți), sunt derivate din clasele Reader si Writer



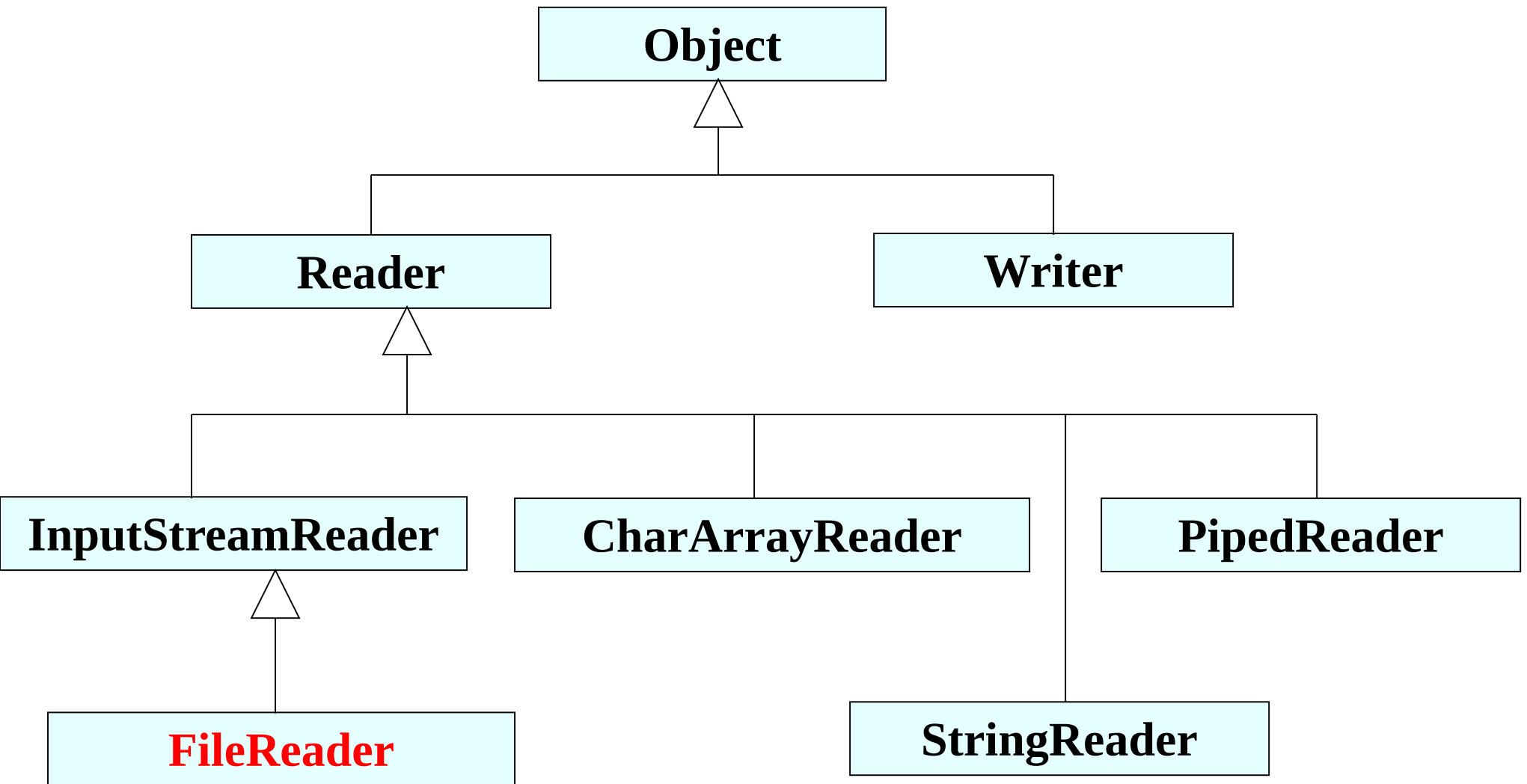
- Dupa actiunea lor:

- ☐ fluxuri primare de citire/scriere a datelor , sau fluxuri de nivel scăzut (se ocupa efectiv cu citirea/scrierea datelor)
- ☐ fluxuri pentru procesarea datelor sau fluxuri de nivel înalt acestea fiind conectate la un flux de nivel scazut pe care îl extinde ca functionalitate cu buffering, filtre pentru conversii de date, etc

Ierarhia fluxurilor de octeti



Ierarhia fluxurilor de caractere



Metode comune fluxurilor

- Superclasele abstracte Reader si InputStream definesc metode similare pentru citirea datelor.

Reader	InputStream
int read() int read(char buf[]) int read(char buf[], int offset,int length)	int read() int read(byte buf[]) int read(byte buf[], int offset,int length)

- Superclasele abstracte Writer si OutputStream sunt de asemenea paralele, definind metode similare pentru scrierea datelor.

Writer	OutputStream
int write() int write(char buf[]) int write(char buf[], int offset,int length)	int write() int write(byte buf[]) int write(byte buf[], int offset,int length)

- Inchiderea oricarui flux se realizeaza prin metoda **close**. In cazul în care aceasta nu este apelata explicit fluxul va fi automat închis când nu va mai exista nici o referinta la el.
Metodele referitoare la fluxuri pot genera exceptii de tipul **IOException**.

Fluxuri de date

- **Buffering** – este o tehnica ce creste performanta lucrului cu fisiere prin scrierea/citirea unui bloc de date si nu doar a unui octet

- Ex:

```
File f = new File("a.txt");
```

```
FileInputStream fs = new  
    FileInputStream(f);
```

```
BufferedInputStream bf = new  
    BufferedInputStream(fs);
```

Fluxuri de date

- **DataInputStream** ofera posibilitatea lucrului cu tipurile de date primitive

- Ex:

```
File f = new File("a.txt");
```

```
FileInputStream fs = new  
    FileInputStream(f);
```

```
BufferedInputStream bf = new  
    BufferedInputStream(fs);
```

```
DataInputStream ds = new  
    DataInputStream(bf);
```

Fluxuri de date

- In continuare se poate citi din fisier cu metode specifice fiecarui tip iar inchiderea fisierului se face prin metoda `close()`

- Ex:

...

```
int i = df.readInt();
```

```
String s = df.readUTF();
```

```
Long l = df.readLong();
```

```
df.close();
```

Fluxuri de date

- Orice flux de date se deschide prin crearea unui obiect (new); fluxurile de nivel inalt primesc la constructor un flux de nivel scazut (inlantuire)
- Se poate citi/scrie in flux cu metode read/write octeti, caractere sau date primitive in functie de tipul fluxului
- Pentru fluxuri de iesire cu buffering metoda flush forteaza scrierea datelor
- Se inchide fluxul cu metoda close()

Fluxuri pentru citirea/scrierea efectiva a datelor

- Clasele ce descriu fluxuri pentru citirea/scrierea efectiva a datelor pot fi împartite în functie de tipul sursei datelor astfel:

Tip sursa	Fluxuri caractere	Fluxuri octeti
Memorie	CharArrayReader, CharArrayWriter	ByteArrayInputStream, ByteArrayOutputStream
	Folosesc pentru scrierea/citirea informatiilor în memorie si sunt create pe un vector existent deja, permițând tratarea vectorilor ca sursa/destinatie pentru crearea unor fluxuri de intrare/iesire	
	StringReader, StringWriter	StringBufferInputStream
	Permit tratarea sirurilor de caractere aflate în memorie ca sursa/destinatie pentru crearea unor fluxuri de intrare/iesire. StringReader si StringWriter sunt folosite cu obiecte de tip String iar StringBufferInputStream cu obiecte de tip StringBuffer.	
Pipe	PipedReader, PipedWriter	PipedInputStream, PipedOutputStream
	Implementeaza componentele de intrare/iesire ale unei conducte de date (pipe). Pipe-urile sunt folosite pentru a canaliza iesirea unui program sau fir de executie catre intrarea altui program sau fir de executie	
Fișier	FileReader, FileWriter	FileInputStream, FileOutputStream
	Numite si <i>fluxuri fisier</i> , acestea sunt folosite pentru citirea datelor dintr-un fisier, respectiv scrierea datelor într-un fisier	

Fluxuri pentru procesarea datelor

- Clasele ce descriu fluxuri pentru procesarea datelor pot fi împartite în funcție de tipul de procesare pe care îl efectuează:

Tip procesare	Fluxuri caractere	Fluxuri octeti
"Bufferizare"	BufferedReader, BufferedWriter	BufferedInputStream, BufferedOutputStream
	Sunt folosite pentru a introduce un buffer în procesul de scriere/citire a informațiilor, reducând astfel numărul de accese la dispozitivul ce reprezintă sursa originală de date. Sunt mult mai eficiente decât fluxurile fără buffer și din acest motiv se recomandă folosirea lor ori de câte ori este posibil	
Filtrare	FilterReader, FilterWriter	FilterInputStream, FilterOutputStream
	Sunt clase abstracte ce definesc o interfață pentru fluxuri care filtrează automat datele citite sau scrise	
Conversie octeti-caractere	InputStreamReader, OutputStreamWriter	
	Formează o punte de legătură între fluxurile de caractere și fluxurile de octeți. Un flux <code>InputStreamReader</code> citește octeți dintr-un flux <code>InputStream</code> și îi convertește la caractere folosind codificarea standard a caracterelor sau o codificare specificată de program. Similar, un flux <code>OutputStreamWriter</code> convertește caractere în octeți și trimite rezultatul către un flux de tipul <code>OutputStream</code>	
Concatenare		SequenceInputStream
	Concatenează mai multe fluxuri de intrare într-unul singur	

Fluxuri pentru procesarea datelor

Tip procesare	Fluxuri caractere	Fluxuri octeti
Conversie tipuri de date		DataInputStream, DataOutputStream
	Folosite la scrierea/citirea datelor de tip primitiv într-un format independent de masina pe care se lucreaza	
Numarare	LineNumberReader	LineNumberInputStream
	Numara liniile citite de la un flux de intrare.	
Citire în avans	PushbackReader	PushbackInputStream
	Fluxuri de intrare care au un buffer de 1-caracter(octet) în care este citit în avans si caracterul (octetul) care urmeaza celui curent citit.	
Afisare	PrintWriter	PrintStream
	Metode convenabile pentru afisarea informatiilor.	

Exemplul 1

- Program care copiază conținutul unui fisier în alt fisier:
- `import java.io.*;`
- `public class Copy {`
- `public static void main(String[] args)`
`throws IOException {`
- `FileReader in = new FileReader("in.txt");`
`FileWriter out = new FileWriter("out.txt");`
- `int c;`
- `while ((c = in.read()) != -1) out.write(c);`
`in.close(); out.close();`
- `}`
- `}`

Exemplul 2

Program care creează un fișier binar prin scrierea unui vector de octeți

```
import java.io.*;
public class ScrieOcteti {
    public static void main(String[] arg) {
        int[] data = { 71, 73, 70, 56, 57, 97, 15, 0, 15, 0,
            128, 0, 0, 255, 255, 255, 0, 0, 0, 44, 0, 0, 0,
            0, 15, 0, 15, 0, 0, 2, 33, 132, 127, 161, 200,
            185, 205, 84, 128, 241, 81, 35, 175, 155, 26,
            228, 254, 105, 33, 102, 121, 165, 201, 145, 169,
            154, 142, 172, 116, 162, 240, 90, 197, 5, 0, 59
        };
        try {
            FileOutputStream fis = new
                FileOutputStream("test.jpg");
            for (int i = 0; i < data.length; i++)
                fis.write(data[i]);
            fis.close();
        } catch (IOException e) {
            System.out.println("Error -- " + e.toString());
        }
    }
}
```

Exemplul 3.

Program care citește un fișier binar și afișează pe ecran conținutul acestuia (octeții)

```
import java.io.*;
public class CitesteOcteti {
    public static void main(String[] arg) {
        try {
            FileInputStream fisier = new
                FileInputStream("desen.jpg");
            boolean eof = false;
            int nr = 0;
            while (!eof) {
                int intrare = fisier.read();
                if (intrare != -1)
                { System.out.print(intrare + " ");
                  nr++; }
                else eof = true;
            }
            fisier.close();
            System.out.println("\nOcteti cititi: " + nr);
        } catch (IOException e) {
            System.out.println("Error -- " + e.toString());
        }
    }
}
```

Exemplul 4.

Program care scrie într-un fișier de date primele 1000 numere prime din șirul numerelor naturale

```
import java.io.*;
class SciePrime {

    public static boolean Prim(int x) {
        double radical = Math.sqrt(x);
        for (int i = 2; i <= radical; i++) {
            if (x % i == 0)
                return false;
        }
        return true;
    }

    public static void main(String arg[]) {
        int[] prime = new int[1000];
        int nrPrime = 0;
        int curent = 2;
        while (nrPrime < 1000) {
            if (Prim(curent)) {
                prime[nrPrime] = curent;
                nrPrime++;
            }
            curent++;
        }

        try {
            FileOutputStream file = new
                FileOutputStream("Nrprime.dat");
            BufferedOutputStream buff = new
                BufferedOutputStream(file);
            DataOutputStream data = new
                DataOutputStream(buff);
            for (int i = 0; i < 400; i++)
                data.writeInt(prime[i]);
            data.close();
        } catch (IOException e) {
            System.out.println("Error -- " +
                e.toString());
        }
    }
}
```

Exemplul 5.

Program care citește un fișier de date ce conține primele 1000 numere prime

```
import java.io.*;
class CitestePrime {
    public static void main(String arg[]) {
        try {
            FileInputStream file = new
                FileInputStream("Nrprime.dat");
            BufferedInputStream buff = new
                BufferedInputStream(file);
            DataInputStream data = new
                DataInputStream(buff);
            try {
                while (true) {
                    int in = data.readInt();
                    System.out.print(in + " ");
                }
            } catch (EOFException eof) {
                buff.close();
            }
        } catch (IOException e) {
            System.out.println("Error -- " + e.toString());
        }
    }
}
```

Exemplul 6.

Program care citește un fișier text și îi afișează conținutul pe ecran. Fișierul text este chiar sursa programului în Java.

```
import java.io.*;
public class CitesteSursa {
    public static void main(String[] arg) {
        try {
            FileReader file = new
                FileReader("CitesteSursa.java");
            BufferedReader buff = new
                BufferedReader(file);
            boolean eof = false;
            while (!eof) {
                String line = buff.readLine();
                if (line == null)
                    eof = true;
                else
                    System.out.println(line);
            }
            buff.close();
        } catch (IOException e) {
            System.out.println("Error -- " + e.toString());
        }
    }
}
```